

Trabajo Final Integrador (TFI)

Aplicación Java con relación 1→1 unidireccional + DAO + MySQL

Objetivo

Desarrollar una aplicación en Java que modele dos clases relacionadas mediante una asociación unidireccional 1 a 1 (la clase "A" referencia a la clase "B"), persistiendo datos en una base relacional mediante JDBC y el patrón DAO, con operaciones transaccionales (commit/rollback) y menú de consola para CRUD.

Consignas generales

- Lenguaje: Java (recomendado 21).
- Persistencia: JDBC (sin ORM) con MySQL.
- Patrón de diseño: DAO y capa Service.
- Código limpio, legible, con manejo de excepciones en todas las capas.
- Relación 1→1 unidireccional: sólo la clase "A" contiene el atributo que referencia a "B".
- Equipos de 4 personas, sin excepción.

Estructura del proyecto (paquetes)

- config/: conexión a la base (lectura de propiedades externas).
- entities/: clases de dominio (A y B) con id y eliminado (baja lógica).
- dao/: interfaces genéricas y DAOs concretos (JDBC + PreparedStatement).
- service/: reglas de negocio, validaciones y orquestación de transacciones.
- main/: arranque de la app y AppMenu (consola).

Requerimientos técnicos

1. Diseño (previo al código)
 - a. Diagrama UML de clases con:
 - i. Atributos (tipo/visibilidad) y métodos (firma/visibilidad).
 - ii. $A \rightarrow B$ (1..1) unidireccional explícita.
 - iii. Paquetes y dependencias principales.
 - b. El UML guía toda la implementación.
2. Entidades (genérico)
 - a. A: atributos propios + id (INT/BIGINT) + eliminado (BOOLEAN). • B: atributos propios + id + eliminado.
 - b. A contiene private B detalle; (o nombre equivalente).
 - c. Constructores (vacío y completo), getters/setters y toString() legible.
3. Base de datos (MySQL)
 - a. Clase DatabaseConnection en config/ con método estático que retorne java.sql.Connection.
 - b. Entregar un archivo .sql con las instrucciones para crear la base de datos y sus tablas (CREATE DATABASE, CREATE TABLE, claves primarias, foráneas, índices y restricciones necesarias para 1→1).
 - c. Entregar otro .sql con datos de prueba (INSERTs) para levantar el proyecto desde cero.
 - d. Relación 1→1 recomendada: clave foránea única en la tabla de B (campo a_id con UNIQUE, FOREIGN KEY a A(id), ON DELETE CASCADE). Alternativa válida: clave primaria compartida (la PK de B es también FK a A).
4. DAO
 - a. GenericDao<T> con: crear(T), leer(long id), leerTodos(), actualizar(T), eliminar(long id).
 - b. DAOs concretos para A y B usando PreparedStatement en todas las operaciones.
 - c. Los DAO deben ofrecer métodos que acepten una Connection externa (para participar de la misma transacción).

5. Service (transacciones obligatorias)
 - a. GenericService<T>: insertar, actualizar, eliminar, getById, getAll.
 - b. AService y BService:
 - i. Abrir transacción: setAutoCommit(false) sobre una conexión compartida.
 - ii. Ejecutar operaciones compuestas (por ej., crear B, asociarla a A y crear A).
 - iii. commit() si todo OK; rollback() ante cualquier error.
 - iv. Restablecer autoCommit(true) y cerrar recursos.
 - v. Validaciones (campos obligatorios, formatos según dominio) y regla 1→1 (impedir más de un B por A).
6. AppMenu (consola)
 - a. Main invoca AppMenu.
 - b. Convertir entradas a mayúsculas donde aplique.
 - c. Debe permitir CRUD completo de A y B: crear, leer por ID, listar, actualizar y eliminar lógico.
 - d. Incluir al menos una búsqueda por un campo relevante (ej.: DNI, dominio, ISBN, etc., según el dominio elegido).
 - e. Manejo robusto de:
 - i. Entradas inválidas (parseos numéricos, formatos).
 - ii. IDs inexistentes.
 - iii. Errores de base de datos y violaciones de unicidad.
 - f. Mensajes claros de éxito/error.

Entregables (obligatorios)

1. Repositorio GitHub público
 - a. Código fuente completo.
 - b. README.md con:
 - i. Descripción del dominio elegido.
 - ii. Requisitos (Java/BD) y pasos para crear la base con el .sql provisto.
 - iii. Cómo compilar y ejecutar (credenciales de prueba y flujo de uso).
 - iv. Enlace al video.
 - c. SQL:
 - i. Archivo con instrucciones para crear la base y tablas.
 - ii. Archivo con datos de prueba.
 - d. UML (imagen .png/.jpg/.pdf).
 - e. Informe (PDF) dentro del repo.

2. Informe (PDF, 6–8 páginas)
 - a. Integrantes (4) y roles.
 - b. Elección del dominio y justificación.
 - c. Diseño: decisiones clave (1→1, FK única vs PK compartida) + UML.
 - d. Arquitectura por capas (responsabilidades de cada paquete).
 - e. Persistencia: estructura de la base, orden de operaciones y transacciones (dónde se hace commit/rollback).
 - f. Validaciones y reglas de negocio.
 - g. Pruebas realizadas (capturas del menú y consultas SQL útiles).
 - h. Conclusiones y mejoras futuras.
 - i. Citar fuentes y herramientas utilizadas (incluida IA, si la usaron).
3. Video (10–15 minutos)
 - a. Presentación de los 4 integrantes con rostro visible.
 - b. Demostración del flujo CRUD y de la relación 1→1 funcionando. o Explicación por secciones de código (entities, dao, service, menú). o Mostrar una operación transaccional y evidenciar el rollback ante un fallo simulado.

Criterios de evaluación

- Correctitud funcional (CRUD, 1→1 unidireccional real en código y base). • Diseño/arquitectura (DAO/Service, responsabilidades claras, validaciones). • Calidad de código (legibilidad, nombres, excepciones, PreparedStatement). • Persistencia (integridad referencial, unicidad para 1→1, scripts SQL reproducibles).
- Documentación (README claro, informe sólido, UML coherente). • Presentación (video dentro del tiempo, explicación técnica, participación equitativa).
- Entrega (repo público completo, proyecto compilable/ejecutable desde cero).

Checklist (sugerido)

- UML actualizado y consistente con el código.
- $A \rightarrow B$ 1 $\rightarrow$ 1 garantizado por FK única o PK compartida. • CRUD de A y B con baja lógica.
- Service orquesta transacciones con commit/rollback.
- DAO acepta Connection externa.
- README.md con pasos para crear base y ejecutar, más enlace al video. • Archivos SQL (creación + datos) incluidos y probados.
- Informe y UML dentro del repo.
- Menú de consola usable y con manejo de errores.

Especificación de campos – TP Integrador (1 $\rightarrow$ 1 unidireccional)

Se listan los campos obligatorios de cada par de clases ($A \rightarrow B$) disponible para el TP Integrador. En todos los casos:

- La relación es unidireccional 1 $\rightarrow$ 1 desde A hacia B (A contiene una referencia a B).
- Ambas clases incluyen id (clave primaria) y eliminado (baja lógica).

Tip: Los nombres y longitudes son sugeridos; podés ajustarlos manteniendo la semántica, unicidad y la relación 1 $\rightarrow$ 1.

Paciente → HistoriaClinica

Clase Paciente (A)

Campo	Tipo (Java)	Reglas / Notas
id	Long	PK
eliminado	Boolean	Baja lógica
nombre	String	NOT NULL, max 80
apellido	String	NOT NULL, max 80
dni	String	NOT NULL, UNIQUE, max 15
fechaNacimiento	java.time.LocalDate	
historiaClinica	HistoriaClinica	Referencia 1→1 a B

Clase HistoriaClinica (B)

Campo	Tipo (Java)	Reglas / Notas
id	Long	PK
eliminado	Boolean	Baja lógica
nroHistoria	String	UNIQUE, max 20
grupoSanguineo	Enum {A+, A-, B+, B-, AB+, AB-, O+, O-}	
antecedentes	String	TEXT
medicacionActual	String	TEXT
observaciones	String	TEXT

Notas generales

- eliminado: usarlo para bajas lógicas (ocultar en listados, no borrar físicamente).
- Validar en Service los campos obligatorios y formatos (email, DNI, ISBN, dominio, CUIT, etc.).